



Università degli Studi di Salerno

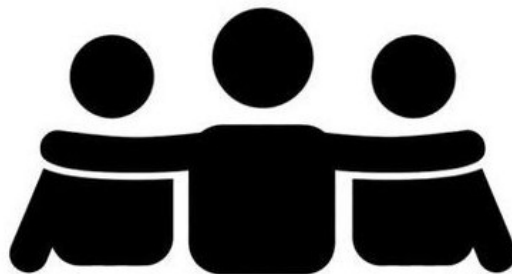
Dipartimento di Informatica

Laurea Triennale in Informatica

Corso di Machine Learning

Social_Matcher

**A Machine Learning Approach to Predict Social
Compatibility Degrees**



Studenti:

Sabetta Francesco

Scarallo Gloria

Matricola:

0512118990

0512119236

Anno Accademico:

2025/2026



Indice

1	Analisi del problema	2
2	Il dataset	3
2.1	Il dataset di partenza	3
2.2	Analisi Esplorativa	4
2.3	Data Cleaning	6
2.4	Feature Engineering	7
2.4.1	Standardizzazione Semantica	7
2.4.2	Preparazione del Dataset	8
2.4.3	Preparazione del Dataset per K-Means (Estrazione Singoli Individui)	8
2.5	Preprocessing	9
2.5.1	XGBoost	9
2.5.2	K-Means	10
3	Scelte di progettazione	12
3.1	XGBoost Regressor	12
3.1.1	Configurazione degli Iperparametri	12
3.1.2	Data Split e Training	13
3.1.3	Utilizzo del modello	13
3.2	K-Means Clustering e Profilazione degli Utenti	14
3.2.1	Il Modello Matematico	14
3.2.2	Scelta del Numero Ottimale di Cluster (k)	14
3.2.3	Addestramento e Serializzazione (Pipeline K-Means)	16
3.2.4	Funzionalità sviluppate	16
4	Analisi delle prestazioni	18
4.1	XGBoost	18
4.2	K-Means	19
4.2.1	Analisi dei Centroidi e Profilazione Psicologica	19
4.2.2	Identificazione dei Cluster	19
4.3	Validità e Significatività dei Cluster Individuati	20
5	Scelte Scartate	21
5.1	Ottimizzazione degli Iperparametri: Transizione alla Nuova Configurazione	21
5.1.1	Analisi della Configurazione Iniziale (Scartata)	21
5.1.2	La Nuova Configurazione	22
6	Conclusioni	23
6.1	Sviluppi Futuri	23

1 Analisi del problema

Le relazioni sociali costituiscono il tessuto fondante della società contemporanea. Che si tratti di scuola, università o lavoro, interagire con gli altri è il modo migliore per evolvere e migliorarci. Ma non tutte le relazioni funzionano in automatico: affinché scatti la scintilla, serve una reale affinità. Il nostro progetto si propone come obiettivo di individuare e stimare la compatibilità tra individui, analizzandone le ambizioni lavorative, gli stili di vita e le sfumature della personalità.

Per affrontare questa sfida in modo esaustivo, il problema è stato modellato seguendo due approcci di Machine Learning distinti e complementari:

- **Apprendimento Supervisionato (Regressione):** In una prima fase, le previsioni sono guidate da un ricco database di casi studio. Analizzando migliaia di coppie, sia affini che incompatibili, il sistema impara a riconoscere gli schemi del successo relazionale. L'obiettivo in questo caso è predire numericamente il grado di affinità, calcolando un valore di `compatibility_score` in una scala da 0 a 100 sulla base di un ampio gruppo di variabili indipendenti.
- **Apprendimento Non Supervisionato (Clustering):** Parallelamente, abbiamo voluto testare un approccio slegato dai punteggi pre-calcolati e fondato sul principio sociologico dell'omofilia, ovvero la tendenza degli individui ad associarsi con i propri simili. Attraverso l'algoritmo K-Means, il sistema raggruppa autonomamente gli utenti in "cluster" psicologici e comportamentali (es. Leader Dinamici, Spiriti Liberi), suggerendo match tra persone che appartengono allo stesso gruppo e condividono la stessa visione della vita.

Questa duplice architettura ci permetterà, in fase di analisi delle prestazioni, di confrontare l'accuratezza predittiva di un modello supervisionato (XGBoost) con le dinamiche esplorative di profilazione di un modello non supervisionato (K-Means).

2 Il dataset

2.1 Il dataset di partenza

Il dataset di partenza, "*Cupid's Dataset*", aveva una diversa idea alla base: calcolare l'affinità romantica tra due individui. Ma attraverso un'analisi del database abbiamo osservato che le variabili a disposizione – con i dovuti e lievi adattamenti – si prestavano in maniera ottimale alla risoluzione di un task di carattere più "generale": la valutazione della compatibilità sociale, estendendone così l'applicabilità ben oltre il solo ambito sentimentale.

Il dataset si compone di 100,000 righe, di cui ciascuna rappresenta una coppia di individui (*Persona A e B*) con 13 features (per ogni persona) che spaziano in 7 categorie:

1. *Personalità*: Sono presi in considerazione diversi tratti caratteriali:
 - **Apertura mentale (openness)**: Cattura la propensione di un individuo verso nuove idee, la sua creatività, l'immaginazione e la volontà di esplorare esperienze inedite. Questo valore contribuisce al calcolo della somiglianza caratteriale: punteggi di openness più vicini tra i due individui della coppia indicano una maggiore compatibilità.
 - **Estroversione (extraversion)**: Riflette la socievolezza, l'assertività, la loquacità e l'energia che un individuo trae dalle interazioni sociali.
 - **Cordialità (agreeableness)**: Riflette la propensione di un individuo verso la compassione, l'empatia, la cortesia e la disponibilità al compromesso. All'interno del dataset presenta una lieve correlazione positiva con l'**extraversion**.
 - **Coscienziosità (conscientiousness)**: Riflette l'autodisciplina di un individuo, il suo senso del dovere, l'affidabilità e la preferenza per comportamenti pianificati e orientati agli obiettivi piuttosto che spontanei.
2. *Carriera*: La carriera (**career_field**), suddivisa esplicitamente in 10 settori (*Tech, Healthcare, Finance, Education, Creative Arts, Law, Engineering, Marketing, Science, Entrepreneurship*), e il livello di ambizione (**career_ambition**), che riflettono l'allineamento e la compatibilità professionale.
3. *Stile di vita*: Il cronotipo (**chronotype**), che determina la predisposizione naturale di un individuo ad essere più attivo ed energico, che ne influenza anche le sue preferenze sui ritmi di sonno e veglia, e la spontaneità (**spontaneity**), che riflette la propensione di un individuo ad agire d'impulso, ad accogliere i cambiamenti dell'ultimo minuto e a vivere il momento, in contrapposizione all'adesione a routine rigide e programmi dettagliati.
4. *Comunicazione*: L'espressività emotiva (**emotional_expressiveness**), che misura quanto apertamente gli individui condividono i propri sentimenti e le proprie emozioni.



5. *Linguaggio dell'amore*: Il linguaggio dell'amore (`love_language`), suddiviso in: Parole di conferma (Words of Affirmation), Atti di servizio (Acts of Service), Ricevere doni (Receiving Gifts), Tempo di qualità (Quality Time) o Contatto fisico (Physical Touch).
6. *Dati demografici*: Includono l'età (`age`, 18–55 anni), il tipo di località (`location_type`, Urbana / Suburbana / Rurale) e il livello di istruzione (`education_level`), strutturato su una scala ordinale a 5 livelli:
 - Diploma Superiore (*High School*)
 - Diploma Universitario (*Associate's*)
 - Laurea Triennale (*Bachelor's*)
 - Laurea Magistrale (*Master's*)
 - Dottorato di Ricerca (*Doctorate*)

Il dataset di partenza prevedeva 3 variabili target, per fare previsioni non solo sul grado di affinità, ma anche la longevità stimata della relazione:

1. **Punteggio di compatibilità** (`compatibility score`): Calcola il grado di affinità tra due individui in una scala da 0 a 100.
2. **Compatibile** (`compatible`): Restituisce 1 (`true`) se gli individui sono compatibili, 0 (`false`) altrimenti. L'utilizzo di questa variabile target avrebbe permesso di modellare il problema come un problema di classificazione binaria.
3. **Longevità della relazione** (`relationship longevity months`): Calcola la longevità della relazione in mesi.

2.2 Analisi Esplorativa

L'analisi esplorativa è stata condotta per comprendere la struttura del dataset grezzo, individuare eventuali anomalie (come i valori mancanti) e valutare le relazioni statistiche tra le variabili prima di procedere alla fase di pulizia e modellazione.

- **Ispezione Strutturale**: Il dataset analizzato è composto da 100.000 record e 30 colonne. Le variabili presentano tre diverse tipologie di dato: 18 variabili numeriche a virgola mobile (`float64`), 6 variabili numeriche intere (`int64`) e 6 variabili categoriali/testuali (`object`).
- **Analisi dei Valori Mancanti (Missing Values)**:

Variabile	Conteggio NaN
a_age	1
a_emotional_expressiveness	1
b_love_language	1

L'ispezione ha rivelato una sostanziale integrità dei dati, con la presenza di un solo valore mancante (*null*) limitato alle variabili `a_age`, `a_emotional_expressiveness` e `b_love_language`. Questa osservazione ha fornito la giustificazione per le successive scelte operative di imputazione basate su media e moda.

- **Analisi della Variabile Target:** Il punteggio di compatibilità (`compatibility_score`) calcola il grado di affinità tra due individui in una scala da 0 a 100. Sulla base di questa variabile target, abbiamo deciso di trattare il problema come un problema di **regressione lineare multipla**. Analizzando la distribuzione della variabile, evince che i suoi valori si concentrano nell'intervallo 60-70.

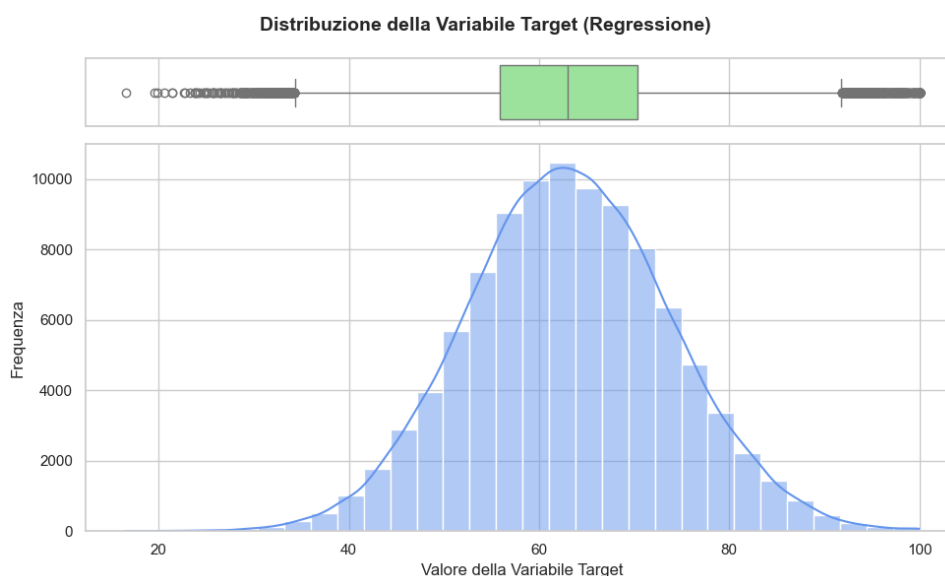


Figura 1: Distribuzione di `compatibility_score`

- **Analisi Statistica Descrittiva:**
 - *Variabili Numeriche:* Evince, nello specifico, che la variabile target principale, ovvero il (`compatibility_score`), presenta una media globale di 63.12 su 100.
 - *Variabili Categoricali:* Per le variabili di tipo testuale, l'area geografica nettamente prevalente per la formazione delle coppie è quella urbana ("Urban"). A livello professionale, i settori lavorativi più frequenti risultano essere la Finanza ("Finance") per la Persona A e il Marketing per la Persona B. Inoltre, i "linguaggi dell'amore" maggiormente ricorrenti nel campione sono "Words of Affirmation" per la Persona A e "Receiving Gifts" per la Persona B.
- **Analisi delle Correlazioni e Importanza delle Feature:**
 - Per esplorare le relazioni tra le variabili indipendenti, e nello specifico la loro importanza e rilevanza nel calcolo delle predizioni, è stato pre-addestrato un modello *Random Forest Regressor* per estrarre l'importanza intrinseca delle feature.

Tabella 1: Feature Importance - Cupid Dataset Analysis

Ranking	Variabile (Feature)	Importanza Relativa
1	a_career_ambition	0.5140
2	b_career_ambition	0.5076
3	b_emotional_expressiveness	0.3978
4	a_emotional_expressiveness	0.3938
5	a_spontaneity	0.1507
6	b_spontaneity	0.1475
7	b_openness	0.1074
8	a_openness	0.1062
9	a_extraversion	0.1028
10	b_extraversion	0.0980
11	b_chronotype	0.0807
12	a_chronotype	0.0763
13	a_conscientiousness	0.0703
14	b_agreeableness	0.0693
15	b_conscientiousness	0.0687
16	a_agreeableness	0.0687
17	b_age	0.0542
18	a_age	0.0539
19	b_career_field	0.0406
20	a_career_field	0.0384
21	b_love_language	0.0245
22	a_love_language	0.0240
23	b_education	0.0200
24	a_education	0.0193
25	b_location	0.0141
26	a_location	0.0135

Dai risultati si evince che il modello è estremamente bilanciato: l'importanza tra Persona A e Persona B è quasi identica. Inoltre, tratti psicologici come la proiezione futura (**career_ambition**) e l'intelligenza emotiva esercitano un'influenza superiore rispetto a variabili strutturali come il livello di istruzione (**education**) o la posizione geografica (**location**). L'analisi ha svelato che i quattro parametri con il maggiore impatto nel determinare il punteggio di compatibilità sono: l'ambizione lavorativa della Persona B (**b_career_ambition**), seguita dall'ambizione della Persona A (**a_career_ambition**) e dall'intelligenza/espressività emotiva di entrambi gli individui (**b_emotional_expressiveness** e **a_emotional_expressiveness**).

2.3 Data Cleaning

La prima fase della pipeline di preparazione dei dati si è concentrata sulla pulizia del dataset grezzo. Sono state eseguite, nello specifico, operazioni di **imputazione** per la gestione dei valori mancanti evidenziati durante la fase di analisi esplorativa.

- **Gestione dei valori mancanti (Imputazione):** L'analisi esplorativa ha evidenziato la presenza di valori *null*. Per evitare la perdita di informazioni associata all'eliminazione delle righe, si è implementata una strategia di imputazione differenziata in base alla natura della singola variabile:
 - Per la variabile `a_emotional_intelligence` (una variabile numerica), i valori mancanti sono stati sostituiti con la **media aritmetica** della distribuzione, calcolata come $\mu = \frac{1}{n} \sum_{i=1}^n x_i$ e arrotondata alla seconda cifra decimale.
 - Per le variabili `a_age` e `b_communication_style`, è stata utilizzata la **moda statistica**.
 - Per assicurare la totale assenza di valori nulli residui, è stata applicata una logica di *fallback* sull'intero dataset: le restanti variabili numeriche (`float64`, `int64`) sono state imputate utilizzando la mediana, misura preferita per la sua robustezza agli eventuali *outlier*; le restanti variabili testuali o categoriche (`object`, `category`) sono state imputate tramite la moda.

2.4 Feature Engineering

Il Feature Engineering è stata una delle fasi più delicate e importanti dell'intero progetto *Social Matcher*. I modelli di Machine Learning, alla fine dei conti, elaborano esclusivamente numeri: se li alimentiamo con dati grezzi, sbilanciati o mal formattati, le loro previsioni saranno inevitabilmente poco accurate, poiché la qualità dei risultati finali dipende strettamente dalla bontà delle informazioni fornite in ingresso.

Questa preparazione dei dati è servita per venire incontro a due necessità architetture molto diverse tra loro. Da una parte, l'algoritmo supervisionato (*XGBoost*) aveva bisogno di informazioni pulite e ben strutturate per riuscire a scovare i pattern nascosti dietro a un match di successo. Dall'altra, il modello non supervisionato (*K-Means*) lavora calcolando le distanze geometriche tra gli utenti: se non avessimo messo tutte le caratteristiche sulla stessa scala di grandezza, una variabile con numeri nativamente più alti avrebbe finito per "schiacciare" matematicamente tutte le altre, falsando completamente i raggruppamenti. L'intervento sui dati, tuttavia, non è stato solamente matematico, ma anche logico e semantico. Il dataset di partenza (*cupid_dataset*) era stato concepito specificamente per il matching romantico. Il nostro obiettivo, invece, era creare uno strumento di social networking più versatile, valido anche per trovare amici o partner lavorativi. Per questo motivo, abbiamo dovuto "tradurre" le feature prettamente romantiche in concetti psicologici più neutri e universali.

Di seguito vengono analizzati nel dettaglio i passaggi metodologici che hanno composto l'intera pipeline di trasformazione, partendo dalle operazioni semantiche comuni a entrambi i modelli per poi scendere nel dettaglio delle preparazioni strutturali specifiche per *XGBoost* e *K-Means*.

2.4.1 Standardizzazione Semantica

Indipendentemente dal modello utilizzato, i dati grezzi sono stati uniformati per slegarli dal contesto puramente "romantico" del dataset originale e adattarli a un contesto di compatibilità sociale (amicale, accademica e lavorativa):

- **Standardizzazione della Nomenclatura:** La variabile *love_language* è stata rinominata in *communication_style*, ed *emotional_expressiveness* in *emotional_intelligence*.
- **Mappatura degli Stili di Comunicazione:** I valori originali sono stati tradotti in stili relazionali generici:
 - *Words of Affirmation* → **Verbal Support**
 - *Quality Time* → **Shared Experiences**
 - *Acts of Service* → **Practical Reliability**
 - *Receiving Gifts* → **Thoughtful Gestures**
 - *Physical Touch* → **Physical Warmth**
- **Discretizzazione dell'Età:** Le variabili continue dell'età sono state trasformate in variabili categoriche ordinali definendo tre range: **18-30, 31-43, 44-55**.
- **Rimozione Variabili Geografiche:** Data la sua scarsa rilevanza emersa durante le analisi preliminari (come visibile in Tabella 1), la variabile *location* è stata eliminata, assumendo che l'affinità caratteriale trascenda la posizione geografica.

2.4.2 Preparazione del Dataset

Per evitare rumore e concentrarsi sulle feature più rilevanti, sono state eliminate:

- *pair_id*: Chiave primaria con potenza predittiva nulla.
- *compatible*: Target binario originale, eliminato in quanto il problema è affrontato come regressione.
- *relationship_longevity_months*: Variabile target non rilevante per questo studio.

Dall'analisi esplorativa si evince che variabili strutturali come il livello di istruzione (*education*), e la posizione geografica (*location*) hanno poca rilevanza per il calcolo delle predizioni sulla variabile target. Abbiamo comunque mantenuto l'istruzione in quanto fondamentale per la profilazione socio-demografica, mentre abbiamo scartato la *location* proprio in virtù del suo scarso peso predittivo. Il risultato di queste operazioni è un nuovo dataset denominato *social_matcher.csv*, pronto per essere dato in pasto all'algoritmo di regressione.

2.4.3 Preparazione del Dataset per K-Means (Estrazione Singoli Individui)

Per applicare il clustering K-Means basato sull'omofilia, l'algoritmo non deve analizzare il successo di una coppia, ma profilare i *singoli utenti*. A tal fine, il dataset originale è stato destrutturato:

1. **Separazione delle colonne:** Le colonne relative alla Persona A e alla Persona B sono state isolate e i relativi prefissi (*a_* e *b_*) rimossi.
2. **Concatenazione Verticale:** I due blocchi sono stati uniti verticalmente, raddoppiando di fatto il numero di righe e trasformando le "coppie" in un database di individui indipendenti.

3. **Rimozione Possibili Duplicati e Target:** Poiché lo stesso individuo poteva apparire in più coppie nel dataset originale, abbiamo rimosso i duplicati per ottenere profili unici. Infine, le colonne target relative alla coppia (es. `compatibility_score`) sono state eliminate, essendo il K-Means un modello di apprendimento non supervisionato.

Il dataset risultante è denominato `adapted_dataset.csv`, il quale verrà usato per lo sviluppo dell'algoritmo K-means.

2.5 Preprocessing

2.5.1 XGBoost

Il modulo *preprocessor* (definito nel file `preprocessing.py` all'interno della directory `utils`) agisce come strato di mediazione (middleware) tra i dati grezzi e l'algoritmo di apprendimento. Il suo scopo principale è garantire che l'input fornito al modello XGBoost sia numericamente stabile e strutturalmente coerente sia in fase di addestramento che di inferenza.

I principali passaggi architetturali sono:

- **Separazione Semantica:** Prima di applicare qualsiasi trasformazione, il codice esegue una separazione semantica del dataset definendo esplicitamente due insiemi di feature:
 - **Variabili Categoriche:** Attributi testuali o discretizzati (come le fasce d'età, il settore lavorativo e lo stile di comunicazione) che necessitano di una conversione in formato numerico.
 - **Variabili Numeriche:** Valori continui o ordinali intrinseci (come i punteggi dei tratti psicologici e il livello di istruzione) che esprimono grandezze già quantificate.
- **Passaggio Diretto dei Valori Numerici (Passthrough):** A differenza di algoritmi spaziali o sensibili alla magnitudo (come il K-Means), XGBoost è un modello basato su alberi decisionali (*tree-based*). Questo significa che l'algoritmo cerca i punti di taglio ottimali valutando l'ordine dei valori e non le loro distanze assolute. Per questo motivo, per le feature numeriche il modulo adotta una strategia di *passthrough*: i dati mantengono la loro scala originale. Si evitano così standardizzazioni matematicamente superflue che non migliorerebbero l'accuratezza, ottimizzando i tempi di calcolo.
- **Encoding Categorico (Ordinal Encoder):** Poiché XGBoost richiede esclusivamente input numerici, le variabili categoriche vengono trasformate tramite **Ordinal Encoding**. Questa tecnica mappa ogni singola categoria (es. "Ingegnere", "Artista") in un numero intero progressivo. Sebbene l'assegnazione di questi numeri possa sembrare introdurre una falsa gerarchia, i modelli *tree-based* avanzati riescono a gestire questa codifica in modo estremamente efficiente, isolando le categorie tramite split successivi. Questo approccio è stato preferito al tradizionale *One-Hot Encoding* per evitare l'esplosione dimensionale del dataset e la conseguente sparsità dei dati.

Il modulo è inoltre progettato per essere tollerante ai guasti (*fault-tolerant*): eventuali categorie sconosciute che dovessero presentarsi in inferenza vengono gestite in modo sicuro assegnando un valore di default pari a -1.

- **Composizione e Filtraggio (Column Transformer):** L'ultima fase consiste nell'assemblare i dati intatti (*passthrough*) e quelli codificati in un'unica entità logica coerente. Sfruttando l'attributo `remainder='drop'`, il trasformatore agisce anche da rigoroso *filtro di sicurezza*: elimina automaticamente dal flusso di dati qualsiasi colonna non esplicitamente dichiarata utile all'interno delle liste iniziali. Questo previene inquinamenti del modello (*data leakage*) e riduce il rumore statistico complessivo.

2.5.2 K-Means

Il modulo *preprocessor* dedicato alla profilazione non supervisionata ricopre un ruolo ancora più critico rispetto alla controparte predittiva. K-Means è un algoritmo spaziale che raggruppa gli utenti calcolando le distanze geometriche (solitamente euclidee) tra i loro profili. Questo lo rende estremamente sensibile alla scala e alla forma dei dati in ingresso.

I passaggi architetturali definiti nel codice sono i seguenti:

- **Separazione Semantica:** Come per il modello supervisionato, il dataset viene esplicitamente diviso. Le variabili categoriche (`age`, `career_field`, `communication_style`) vengono isolate da quelle numeriche, che, in questo caso, comprendono l'istruzione e tutti i tratti continui legati alla personalità e all'attitudine (`openness`, `extraversion`, ecc.).
- **Feature Scaling (StandardScaler):** A differenza degli alberi decisionali, qui il passaggio diretto dei numeri non è applicabile. Se una variabile avesse un range di valori molto più ampio delle altre, dominerebbe matematicamente il calcolo della distanza, "schiacciando" l'influenza dei restanti tratti psicologici. Per risolvere questo problema, viene applicato lo **StandardScaler**, che trasforma tutte le feature numeriche in modo che abbiano una media (μ) pari a 0 e una varianza (σ^2) pari a 1. In questo modo, ogni singola sfaccettatura della personalità ha esattamente lo stesso peso spaziale durante il calcolo dei centroidi.
- **Encoding Categorico (One-Hot Encoder):** L'Ordinal Encoding usato per XGBoost risulterebbe disastroso per il K-Means: assegnare numeri come 1, 2 e 3 a professioni diverse farebbe credere all'algoritmo che la professione 2 si trovi geometricamente "a metà strada" tra la 1 e la 3. Per evitarlo, si utilizza il **One-Hot Encoding**, che divide ogni categoria in una colonna binaria (0 o 1) indipendente. Questo garantisce che tutte le categorie siano vettorialmente ortogonali ed equidistanti tra loro. Il trasformatore è stato inoltre impostato con `sparse_output=False` per generare matrici dense compatibili con le pipeline successive, e con `handle_unknown='ignore'` per far sì che l'applicazione web non vada in crash qualora un utente in fase di inferenza dovesse inserire un dato non previsto.



- **Composizione (Column Transformer):** L'assemblaggio finale unisce i tratti psicologici standardizzati e le categorie vettorializzate in un'unica matrice omogenea, pronta per essere proiettata nello spazio n-dimensionale per la ricerca dei cluster.

3 Scelte di progettazione

Abbiamo deciso di implementare e mettere a confronto due modelli basati su due approcci diversi, uno supervisionato con il modello *XGBoost Regressor*, e uno non supervisionato con l'algoritmo *k-means*.

3.1 XGBoost Regressor

Per il calcolo del punteggio di compatibilità si è scelto di implementare il modello **XGBoost** (**XGBRegressor**), un algoritmo avanzato basato sul *Gradient Boosting*. La progettazione del sistema di predizione del `compatibility_score` è stata guidata dai principi di *robustezza, modularità e prevenzione dell'overfitting*. Rispetto ad altri modelli ad albero, XGBoost offre prestazioni superiori, una migliore gestione dei dati tabulari complessi e un controllo estremamente granulare sulla regolarizzazione, fondamentale per un dataset basato su tratti psicologici. Grazie anche all'uso integrato della regolarizzazione L1 (Lasso) e L2 (Ridge) all'interno dell'algoritmo stesso, è molto più facile evitare che il modello impari a memoria il training set rispetto ad altri algoritmi.

3.1.1 Configurazione degli Iperparametri

Il modello è stato configurato con l'idea di bilanciare due esigenze: prevenire l'overfitting, evitando che il modello "impari a memoria" il dataset di addestramento, e garantire l'apprendimento anche su casi "limite", evitando che concentrasse le sue predizioni in un intervallo troppo ristretto.

- **Flessibilità e Cattura dei Casi Estremi:**

- `max_depth=6`: Consentendo la costruzione di alberi profondi fino a 6 livelli, il modello è in grado di intercettare incroci complessi tra le variabili (es. dinamiche tra età, settore lavorativo e tratti caratteriali) necessari per prevedere accuratamente i picchi di alta o bassa compatibilità.
- `min_child_weight=2`: Questo valore rappresenta un bilanciamento ideale. Permette al modello di imparare anche da piccoli gruppi di profili con punteggi rari, esigendo tuttavia un minimo di 2 casi simili per confermare la regola ed evitare così di memorizzare singole anomalie isolate.
- `gamma=0.0`: Impostando questo parametro a zero, si consente all'albero di ramificare liberamente per scovare pattern sottili nei dati, senza subire potature preventive.

- **Apprendimento Stabile:**

- `n_estimators=250` e `learning_rate=0.05`: Un numero di alberi proporzionato, unito a un tasso di apprendimento contenuto, garantisce al modello il tempo e lo spazio necessari per apprendere regole complesse, procedendo in modo lento e costante per minimizzare gli errori globali.
- `subsample=0.85` e `colsample_bytree=0.85`: Ogni albero viene addestrato sull'85% dei dati e sull'85% delle feature. Questa visione ampia e completa

infonde grande "sicurezza" statistica al modello, permettendogli di sbilanciarsi verso punteggi più netti, pur mantenendo un 15% di casualità fondamentale per prevenire l'overfitting.

- **Regularizzazione Matematica Bilanciata:**

- Si fa uso di una regularizzazione **L1** leggera (`reg_alpha=0.1`) per spegnere il "rumore" di fondo delle feature ininfluenti, affiancata da una **L2** standard (`reg_lambda=1.0`). Questa impostazione controlla i pesi decisionali senza schiacciare artificialmente i punteggi verso il valore medio globale, consentendo alle previsioni di distribuirsi in modo fluido e realistico su tutto il range.

3.1.2 Data Split e Training

Il modello viene addestrato con un approccio *off-line learning* (eseguito in un'unica sessione pre-calcolata) utilizzando il dataset pulito *social_matcher.csv*. Per permettere l'addestramento, la variabile target (`compatibility_score`) viene isolata dalle feature predittive. Il dataset viene successivamente suddiviso in un training set (**80%**) e un test set (**20%**) tramite tecniche standard di hold-out, impostando un seed pseudocasuale fisso (`random_state=42`) al fine di garantire la totale riproducibilità dell'esperimento in esecuzioni future.

Un elemento architetturale cruciale dello script è l'implementazione di una *Pipeline*. Questa struttura non si limita al mero addestramento dell'algoritmo, ma incapsula e concatena in un unico flusso sequenziale sia le operazioni di **preparazione dei dati** (importate dal modulo `build_xgb_preprocessor`), sia l'istanza configurata dell'algoritmo XGBoost (`get_xgb_model`). Questo design pattern garantisce l'assenza di data leakage e assicura che le medesime trasformazioni applicate durante il training vengano eseguite automaticamente, e in modo coerente, su qualsiasi nuovo dato in fase di inferenza.

Infine, la pipeline completa viene serializzata nel file `social_matcher_model.pkl` utilizzando la libreria *joblib*. I dati di test (`X_test.pkl` e `y_test.pkl`) vengono salvati separatamente in un'apposita directory di risorse. Questa strategia di persistenza disaccoppia in modo netto la fase di addestramento da quelle future di validazione e produzione, consentendo di importare il "cervello" del sistema in qualsiasi applicativo esterno senza dover ripetere il dispendioso processo di apprendimento.

3.1.3 Utilizzo del modello

Il programma `app.py` rappresenta l'**interfaccia grafica** che permette di valutare la compatibilità tra due individui. Il programma presenta una schermata chiara dove è possibile inserire le caratteristiche di due persone distinte (Profilo A e Profilo B), specificando informazioni quali l'età, il livello di istruzione, la professione e i tratti salienti del carattere. Premendo il pulsante "Calcola Affinità", il sistema raccoglie i dati inseriti, interroga il modello e restituisce la sua predizione, ovvero una percentuale di compatibilità (da 0% a 100%).

3.2 K-Means Clustering e Profilazione degli Utenti

Mentre XGBoost si concentra sulla predizione diretta dell'affinità di coppia basandosi su dati storici, il secondo approccio adottato si basa sull'apprendimento non supervisionato (*unsupervised learning*) tramite l'algoritmo **K-Means**.

L'obiettivo in questa fase non è prevedere un punteggio precalcolato, bensì scoprire la struttura intrinseca dei dati per raggruppare i singoli individui in profili psicologici e comportamentali distinti. L'ipotesi sociologica alla base di questo approccio è l'**omofilia**: la tendenza naturale delle persone a formare legami forti e duraturi con individui simili a loro.

3.2.1 Il Modello Matematico

L'algoritmo K-Means partiziona il dataset in k cluster predefiniti, cercando di minimizzare la varianza interna ai gruppi. La metrica ottimizzata durante l'addestramento è la somma dei quadrati delle distanze all'interno dei cluster (*Within-Cluster Sum of Squares* o **Inerzia**), definita matematicamente come:

$$Inerzia = \sum_{j=1}^k \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$

dove k è il numero di cluster, x_i è il vettore delle feature del singolo utente, C_j è il cluster di appartenenza e μ_j è il centroide (il "profilo medio" o baricentro) del cluster j .

3.2.2 Scelta del Numero Ottimale di Cluster (k)

Poiché l'algoritmo K-Means richiede di specificare il numero di cluster a priori, abbiamo utilizzato due delle metriche più robuste in letteratura per determinare il valore ottimale di k in modo oggettivo, testando configurazioni da 2 a 10 cluster:

- **Metodo del Gomito (Elbow Method):** Abbiamo analizzato l'andamento dell'Inerzia all'aumentare di k . Poiché l'Inerzia diminuisce fisiologicamente aggiungendo nuovi centroidi, l'obiettivo è stato individuare il punto di flesso (il "gomito") nella curva, ovvero il valore di k oltre il quale l'aggiunta di ulteriori cluster non porta a una diminuzione significativa e giustificabile della varianza interna.

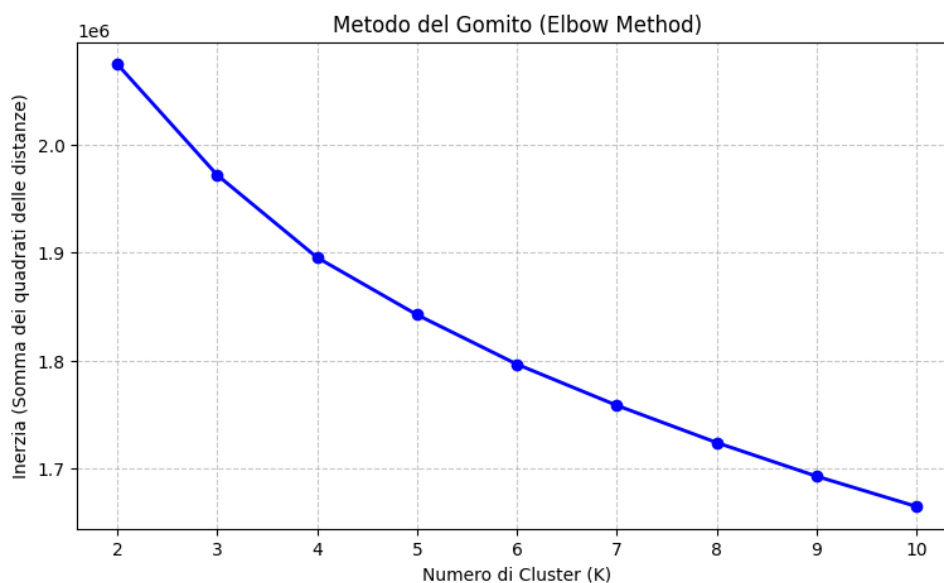


Figura 2: Metodo del Gomito (Elbow Method) per la minimizzazione dell’Inerzia.

- **Silhouette Score:** Per validare la scelta del gomito, abbiamo calcolato il coefficiente di Silhouette, che valuta la densità e la separazione dei cluster.

È fondamentale sottolineare un dettaglio tecnico: il calcolo del Silhouette Score ha una complessità computazionale quadratica, pari a $\mathcal{O}(N^2)$. Calcolare le distanze di ogni singolo utente rispetto a tutti gli altri membri del dataset completo avrebbe saturato la memoria RAM e richiesto tempi di calcolo insostenibili. Per ovviare a questo limite architetturale, abbiamo adottato la pratica standard di calcolare la metrica su un sottocampione casuale di 50.000 istanze, garantendo al contempo un’assoluta valenza statistica e la fattibilità computazionale.

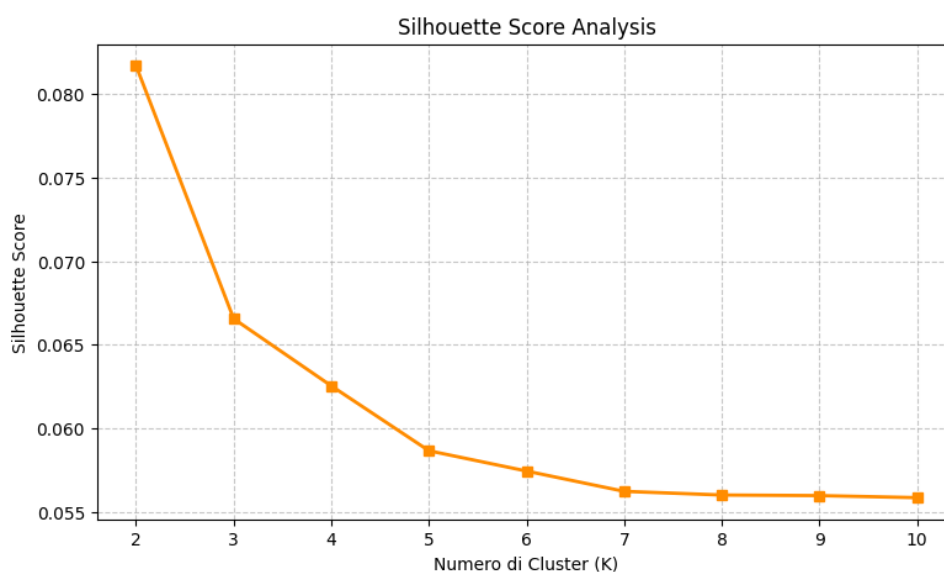


Figura 3: Andamento del Silhouette Score al variare di k .

Dall'incrocio dei grafici (Figura 2 e Figura 3), è emerso in modo chiaro che il numero ottimale per segmentare la nostra utenza senza creare frammentazioni forzate è $k = 4$.

3.2.3 Addestramento e Serializzazione (Pipeline K-Means)

Una volta definiti i passaggi di trasformazione matematica dei dati, si è proceduto con la fase di addestramento vera e propria del modello non supervisionato. L'architettura del codice di training è stata progettata seguendo le *best practice* dell'ingegneria del software per il Machine Learning, focalizzandosi su modularità, riproducibilità e prontezza per il deployment.

I punti chiave di questa fase di implementazione sono:

- **Integrazione tramite Pipeline:** Invece di eseguire il preprocessing e l'addestramento come due passaggi separati e manuali, l'intero flusso è stato incapsulato all'interno di un oggetto **Pipeline** della libreria Scikit-Learn. Questa struttura concatena sequenzialmente il trasformatore (**preprocessor**) e l'algoritmo (**KMeans**). Questo approccio non solo rende il codice più elegante e leggibile, ma garantisce che le medesime trasformazioni applicate al dataset di addestramento vengano applicate in modo identico e automatico ai nuovi dati in fase di inferenza.
- **Configurazione degli Iperparametri:** L'istanza dell'algoritmo K-Means è stata inizializzata con parametri specifici per garantirne la stabilità:
 - **n_clusters=4:** Il numero di centroidi da individuare, derivato empiricamente dalle analisi preventive di Inerzia e Silhouette.
 - **n_init=10:** Il numero di inizializzazioni indipendenti con centroidi di partenza diversi. L'algoritmo esegue 10 corse complete e seleziona quella con l'inerzia minore, riducendo drasticamente il rischio di convergere verso ottimi locali sub-ottimali.
 - **random_state=42:** L'impostazione di un "seme" fisso per il generatore di numeri pseudo-casuali, fondamentale per garantire la totale riproducibilità scientifica dell'esperimento a parità di dataset.
- **Serializzazione del Modello (Joblib):** Al termine dell'addestramento sul dataset pre-elaborato, l'intera **Pipeline** è stata serializzata e salvata su disco in formato **.pkl** tramite la libreria *joblib*. Risulta fondamentale sottolineare che, avendo esportato la Pipeline e non il solo modello, il file risultante (**kmeans_model.pkl**) contiene sia la "memoria spaziale" dei centroidi sia le regole di standardizzazione ed encoding. In questo modo, l'applicazione web finale è in grado di ingerire i dati grezzi inseriti dall'utente senza necessitare di ulteriori script di formattazione intermedi.

3.2.4 Funzionalità sviluppate

A valle della fase di addestramento e profilazione, il modello K-Means definitivo è stato applicato all'intero bacino di utenti presente nel file **adapted_dataset.csv**. Per ciascun individuo, l'algoritmo ha calcolato la distanza dai quattro centroidi e ha predetto la relativa etichetta di appartenenza. Questa informazione è stata integrata aggiungendo



una nuova colonna (`cluster`) al dataframe, il quale è stato infine esportato e salvato nel nuovo file `clustered_dataset.csv`.

Questo passaggio architetturale è fondamentale, in quanto `clustered_dataset.csv` funge da vero e proprio database storicizzato e pre-calcolato per l'applicazione interattiva finale. Sviluppata in Python tramite il framework Streamlit, l'applicazione sfrutta sia il modello serializzato (`kmeans_model.pkl`) che questo database strutturato per offrire due diverse modalità di interazione:

- **Confronto di Gruppo:** L'utente inserisce manualmente i dati di un gruppo ristretto di individui (fino a 5 persone). L'app interroga il modello in tempo reale per calcolarne istantaneamente i rispettivi cluster e verificare la presenza di un'affinità diretta (omofilia) all'interno del gruppo.
- **Ricerca nel Database (Omofilia):** L'utente inserisce esclusivamente il proprio profilo psicologico e demografico. Il motore di matching calcola la sua distanza geometrica dai 4 centroidi per assegnargli un cluster. Successivamente, interrogando il file `clustered_dataset.csv`, estrae automaticamente dal database i profili pre-calcolati più affini, suggerendo partner o colleghi con la medesima struttura psicologica.

4 Analisi delle prestazioni

4.1 XGBoost

Per valutare le performance del regressore lineare, si è scelto di calcolare sia il **RMSE** (*Root Mean Squared Error*) che il **MAE** (*Mean Absolute Error*). Queste due metriche rappresentano lo standard d'eccellenza per i problemi di regressione continua, in quanto misurano in modo complementare la distanza tra le previsioni dell'algoritmo e i valori reali del `compatibility_score`.

- **MAE (Mean Absolute Error):** Calcola la media degli scarti assoluti tra le previsioni del modello ($\hat{y}_i$) e i valori reali (y_i).

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Il MAE fornisce una misurazione lineare e altamente intuitiva dell'errore medio, ed è una metrica robusta rispetto agli *outlier* (valori anomali).

- **RMSE (Root Mean Squared Error):** Rappresenta la radice quadrata della media degli errori al quadrato.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Elevando l'errore al quadrato prima di calcolarne la media, il RMSE penalizza in modo quadratico (quindi molto più severo) gli errori di grande entità. Nel contesto di un algoritmo di "matching", prevedere una compatibilità eccellente per una coppia totalmente incompatibile è un errore critico che il modello deve evitare; il RMSE evidenzia e penalizza fortemente queste gravi discrepanze molto meglio del MAE.

- **Risultati:** I risultati della fase di test evidenziano un valore di **MAE** pari a 7.44; ciò indica che, in media, le previsioni generate dall'algoritmo si discostano dal reale punteggio di compatibilità di circa 7.44 punti.

Parallelamente, il modello ha registrato un **RMSE** pari a 9.34. Il fatto che l'RMSE sia solo moderatamente superiore al MAE suggerisce una distribuzione degli errori relativamente omogenea e una presenza limitata di scostamenti di grossa entità (*outlier*), i quali verrebbero altrimenti penalizzati in modo esponenziale nel calcolo dell'RMSE. Nel complesso, considerando la complessità intrinseca nel prevedere le dinamiche relazionali umane, queste metriche indicano una capacità predittiva solida e soddisfacente.

L'utilizzo combinato di MAE e RMSE consente una diagnostica avanzata del modello. Poiché matematicamente il RMSE sarà sempre maggiore o uguale al MAE, analizzare il divario tra i due valori permette di capire la distribuzione degli errori. Se il RMSE è significativamente più alto del MAE, significa che il modello produce alcuni errori isolati ma molto gravi. Se i due valori sono prossimi, l'errore del modello è costante e omogeneo su tutte le previsioni.

4.2 K-Means

4.2.1 Analisi dei Centroidi e Profilazione Psicologica

L'addestramento del modello spaziale definitivo con 4 cluster ha prodotto risultati sociologicamente e psicologicamente molto coerenti, delineando macro-profilati netti. Per tradurre le coordinate puramente matematiche in descrizioni umane, abbiamo estratto i centroidi finali: di seguito riportiamo le medie per i tratti continui (Tabella 2) e le mode per i tratti categoriali (Tabella 3).

Tabella 2: Medie delle feature numeriche e psicologiche per Cluster

Feature	Cluster 0	Cluster 1	Cluster 2	Cluster 3
Ambizione	0.38	0.59	0.39	0.63
Apertura	0.50	0.50	0.50	0.50
Estroversione	0.41	0.67	0.56	0.36
Amicalità	0.46	0.59	0.53	0.43
Coscienziosità	0.64	0.58	0.34	0.44
Cronotipo	0.57	0.54	0.42	0.47
Spontaneità	0.39	0.43	0.62	0.55
Int. Emotiva	0.43	0.63	0.54	0.40

Tabella 3: Moda delle feature categoriali e demografiche per Cluster

Cluster	Età	Settore	Stile Com.	Istruzione
0	31-43	Law	Thoughtful Gestures	Diploma Univ. (Associate's)
1	31-43	Entrepreneurship	Shared Experiences	Laurea Triennale (Bachelor's)
2	31-43	Education	Practical Reliability	Diploma Univ. (Associate's)
3	31-43	Creative Arts	Thoughtful Gestures	Laurea Magistrale (Master's)

4.2.2 Identificazione dei Cluster

Incrociando i dati statistici raccolti dalle tabelle, è stato possibile "dare un volto" alle distanze matematiche, etichettando semanticamente i 4 gruppi psicologici individuati dall'algoritmo:

- **Cluster 0 – Gli Strutturati / I Tradizionalisti:** Presentano un'alta coscienziosità (0.64) associata alla più bassa propensione alla spontaneità (0.39) del dataset. Lavorano prevalentemente nel settore legale (*Law*), un ambito che per sua natura richiede enorme rigore. Rappresentano individui amanti delle regole e della stabilità,

con un'ambizione di carriera moderata (0.38) e un forte apprezzamento per i gesti premurosi (*Thoughtful Gestures*).

- **Cluster 1 – I Leader Dinamici:** Sono caratterizzati dai punteggi più alti in assoluto per estroversione (0.67), intelligenza emotiva (0.63) e amicalità (0.59). Sono individui estremamente socievoli e la loro naturale propensione all'imprenditoria (*Entrepreneurship*) si allinea perfettamente a un'altissima ambizione lavorativa (0.59). Dal punto di vista relazionale, comunicano e legano prevalentemente attraverso la condivisione attiva del tempo (*Shared Experiences*).
- **Cluster 2 – Gli Spiriti Liberi / I Rilassati:** In questa categoria spicca un punteggio eccezionalmente basso nella coscienziosità (0.34) contrapposto alla più alta spontaneità dell'intero campione analizzato (0.62). Affini al mondo dell'istruzione e della formazione (*Education*), questi individui rifuggono la programmazione ossessiva, preferendo un approccio flessibile alla vita. Nelle relazioni interpersonali cercano soprattutto supporto e affidabilità pratica (*Practical Reliability*).
- **Cluster 3 – Gli Intellettuali Introversi:** Questo gruppo presenta una combinazione molto peculiare: detiene la massima ambizione lavorativa in assoluto (0.63) e il più alto grado di istruzione (*Master's Degree*), ma registra i valori più bassi sia per estroversione (0.36) che per intelligenza emotiva (0.40). Spesso impegnati in ambiti creativi o intellettuali (*Creative Arts*), si configurano come profili focalizzati in modo quasi esclusivo sui propri obiettivi, ma tendenzialmente molto riservati e chiusi nella sfera sociale.

4.3 Validità e Significatività dei Cluster Individuati

La divisione del campione in quattro macro-gruppi non rappresenta solo un traguardo matematico, ma si dimostra una scelta solida e sensata anche dal punto di vista logico e applicativo. La validità di questo risultato può essere valutata su due aspetti principali:

- **Coerenza Psicologica:** L'algoritmo non ha semplicemente raggruppato i dati in modo casuale, ma ha fatto emergere profili realistici che richiamano modelli psicologici noti. Per esempio, la netta contrapposizione tra il Cluster 0 (alta coscienziosità e bassa spontaneità) e il Cluster 2 (bassa coscienziosità e alta spontaneità) riflette dinamiche relazionali molto comuni nella realtà. Allo stesso modo, il modello è riuscito a separare l'estroversione tipica dei leader (Cluster 1) dall'ambizione più solitaria degli intellettuali (Cluster 3). Il fatto che il K-Means, lavorando in modo del tutto non supervisionato, abbia ricostruito profili umani così coerenti dimostra in modo oggettivo l'efficacia del lavoro di preparazione dei dati (Feature Engineering) svolto a monte.
- **Equilibrio del Modello (Parsimonia):** Dal punto di vista tecnico, la scelta di quattro gruppi rappresenta il bilanciamento ideale tra precisione descrittiva e utilità generale del sistema. Un numero inferiore di cluster (es. $k = 2$) avrebbe banalizzato l'analisi, riducendola a distinzioni troppo elementari (come la semplice dicotomia tra introversi ed estroversi). Al contrario, forzare un numero molto più alto (es. $k = 8$) avrebbe frammentato eccessivamente i dati, creando gruppi troppo piccoli e

specifici. Questa eccessiva frammentazione (assimilabile al problema dell'*overfitting*) avrebbe reso il sistema troppo rigido, compromettendo la capacità dell'algoritmo di classificare e associare correttamente nuovi utenti in futuro.

In definitiva, l'architettura a quattro cluster si rivela essere la soluzione più bilanciata. Essa riesce a catturare la reale complessità delle dinamiche relazionali umane mantenendo una struttura matematica pulita, generalizzabile e facilmente applicabile a nuovi campioni di utenti.

5 Scelte Scartate

5.1 Ottimizzazione degli Iperparametri: Transizione alla Nuova Configurazione

Durante la fase di validazione e testing del modello **XGBoost Regressor**, è emersa la necessità di revisionare gli iperparametri iniziali. Sebbene la prima configurazione garantisse una forte protezione contro l'*overfitting*, limitava eccessivamente le capacità predittive dell'algoritmo, schiacciando le previsioni in un intervallo troppo ristretto che andava circa dal 60 al 70 %.

5.1.1 Analisi della Configurazione Iniziale (Scartata)

Il modello di partenza era stato configurato con un approccio estremamente conservativo. Le metriche di valutazione sul Test Set registravano un **Errore Medio Assoluto (MAE) di 7.68** e un **Root Mean Squared Error (RMSE) di 9.65**.

Nonostante l'errore fosse numericamente accettabile, l'analisi della distribuzione delle previsioni ha rivelato un grave problema di **regressione verso la media**: il modello tendeva a comprimere quasi tutte le previsioni all'interno di un ristretto range tra 60 e 70 punti. A causa di vincoli troppo stringenti (come `min_child_weight=5` e `max_depth=4`), l'algoritmo ignorava sistematicamente le casistiche che portavano ad affinità estreme (es. > 80 o < 40), classificandole come rumore statistico e preferendo rifugiarsi in previsioni medie per minimizzare il rischio di errore.

Di seguito riportiamo la vecchia configurazione del modello:

Listing 1: Configurazione iniziale di XGBRegressor

```
XGBRegressor(  
    n_estimators=200,  
    learning_rate=0.05,  
    max_depth=4,  
    min_child_weight=5,  
    gamma=0.1,  
    subsample=0.7,  
    colsample_bytree=0.7,  
    reg_alpha=0.1,  
    reg_lambda=1.0,  
    n_jobs=-1,
```

```
random_state=42,  
eval_metric='rmse'  
)
```

5.1.2 La Nuova Configurazione

L'obiettivo della nuova configurazione è stato quello di "sbloccare" il range predittivo, permettendo al modello di esplorare punteggi molto alti o molto bassi, pur mantenendo una solida stabilità matematica per impedire che il modello impari "a memoria" dai dati (overfitting).

- **Aumento della Capacità di Apprendimento:**

- **n_estimators:** Incrementato da **200 a 250**. Dà al modello un margine maggiore di alberi per correggere gli errori residui.
- **max_depth:** Incrementato da **4 a 6**. Gli alberi più profondi consentono di intercettare incroci complessi tra più feature (es. dinamiche specifiche tra età, carriera e tratti caratteriali), fondamentali per identificare i "perfect match".

- **Cattura delle Anomalie ed Estremi:**

- **min_child_weight:** Ridotto drasticamente da **5 a 2**. È la modifica più impattante. Il valore 5 costringeva il modello a ignorare le regole supportate da poche coppie, penalizzando le affinità rare. Il nuovo valore rappresenta lo *sweet spot*: permette di esplorare i picchi predittivi, esigendo però almeno 2 coppie simili a garanzia della bontà della regola.
- **gamma:** Azzerato (da **0.1 a 0.0**). Rimuove la potatura preventiva, lasciando l'albero libero di ramificare per scovare dettagli sottili.

- **Aumento della Fiducia Statistica:**

- **subsample e colsample_bytree:** Incrementati da **0.7 a 0.85**. Aniché usare il 70% dei dati, ogni albero ha ora accesso all'85% del dataset e delle caratteristiche. Fornendo una visione più ampia e sicura, si incoraggia l'algoritmo a prendere decisioni nette e sbilanciarsi sui punteggi estremi, mantenendo comunque un 15% di stocasticità contro l'overfitting.

- **Mantenimento dei Freni Matematici:**

- **reg_alpha=0.1** e **reg_lambda=1.0**, così come il **learning_rate=0.05**, sono stati mantenuti invariati. Avendo allentato i vincoli strutturali degli alberi, si è scelto di mantenere inalterata la penalità matematica di base per controllare i pesi decisionali senza schiacciare le previsioni.

Inoltre, il miglioramento non è avvenuto a scapito della precisione globale, ma ha addirittura abbassato il **Root Mean Squared Error (RMSE) a 9.34**, dimostrando che il modello sbloccato non solo è più dinamico, ma commette mediamente meno errori rispetto alla configurazione iper-conservativa precedente.

6 Conclusioni

Il lavoro ha avuto come obiettivo principale lo sviluppo di modelli predittivi in grado di individuare pattern complessi legati all'affinità tra individui, con lo scopo di agevolare la formazione di gruppi di studio, team di lavoro e reti sociali. Dall'analisi empirica dei risultati emerge che entrambi i modelli proposti si rivelano validi, robusti e ben bilanciati, garantendo previsioni accurate sull'affinità di una coppia o di un gruppo con un margine di errore decisamente contenuto, con un basso numero di errori "gravi".

Il focus del progetto è stata la fase di **Data Engineering**. Il dataset originale è stato accuratamente pulito da rumore e valori mancanti, per poi procedere con una selezione mirata delle feature più rilevanti per la risoluzione del problema. I dati sono stati successivamente elaborati attraverso due pipeline di pre-processing distinte, appositamente ottimizzate per le specifiche esigenze architetturali di ciascun modello.

Il progetto è stato sviluppato sfruttando gli strumenti offerti da Python, che hanno permesso di gestire in modo efficiente la preparazione dei dati, l'addestramento degli algoritmi e, soprattutto, la rigorosa valutazione delle performance. La validazione dei risultati si è basata su metriche consolidate per i due modelli: la Mean Absolute Error (MAE) e la Root Mean Squared Error (RMSE) per l'algoritmo di regressione, e il Coefficiente di Silhouette per l'analisi di clustering tramite K-Means.

Infine, la realizzazione dell'applicazione interattiva ha rappresentato il vero punto di arrivo del progetto. Questa interfaccia dimostra concretamente come i modelli matematici sviluppati "dietro le quinte" possano essere trasformati in uno strumento pratico, intuitivo e immediatamente utilizzabile da qualsiasi utente.

6.1 Sviluppi Futuri

Di seguito riportiamo alcune possibili estensioni, espansioni e miglioramenti del nostro progetto attuale, attraverso la sperimentazione di diverse tecniche e modelli di previsione.

- **Implementazione di algoritmi avanzati (Deep Learning):** Sebbene gli attuali modelli abbiano mostrato ottime prestazioni, sviluppi futuri potrebbero esplorare l'utilizzo di reti neurali profonde (es. Graph Neural Networks) per catturare relazioni non lineari e pattern di interazione sociale ancora più complessi.
- **Integrazione di dati testuali tramite NLP:** Il sistema attuale potrebbe essere esteso tramite la raccolta di dati testuali (es. bio, interessi descritti a parole, o scambi di messaggi): un futuro step potrebbe prevedere l'uso del Natural Language Processing per estrarre informazioni semantiche e arricchire ulteriormente le feature a disposizione.
- **Sistemi di Feedback e Active Learning:** Un'evoluzione naturale dell'applicativo consisterebbe nell'introdurre un meccanismo di feedback (es. "pollice in su / pollice in giù" sui match suggeriti). Questo permetterebbe di implementare logiche di *Active Learning*, in cui il modello XGBoost si riaddestra periodicamente e dinamicamente imparando dalle interazioni reali degli utenti.